

LASSI-Plus: haplotype frequency spectrum analysis for selective sweeps

Zachary A. Szpiech, PhD

Michael DeGiorgio, PhD

v1.3.1

Contents

1	Introduction	2
2	Installation	2
3	The two stages	2
4	Input files	3
4.1	VCF	3
4.2	Population file	3
4.3	Genetic map	3
5	Statistics	3
6	Missing data	4
7	Reproducibility	5
8	Command line options	5
8.1	General	5
8.2	Input and output	5
8.3	Windows	6
8.4	Statistics	6
8.5	Filtering	7
8.6	saltiLASSI	8
9	Output files	8

10 Exit codes	9
11 Examples	9
12 References	10

1 Introduction

LASSI-Plus implements several haplotype frequency spectrum statistics for detecting and classifying hard and soft selective sweeps, together with the H and G haplotype statistics they build on. Source code, precompiled binaries, and example data are available at <https://github.com/szpiech/lassip>.

The option reference in section 8 is generated from the program's own `--help` output, so it describes the binary this manual was built against rather than a past release.

2 Installation

```
cd src && make
```

The only requirements are a C++11 compiler and zlib. `make check` runs the regression suite in `tests/`, which compares output against recorded files and takes about a minute. `src/Makefile` builds with `-O3` and no architecture specific flags; set `ARCHFLAGS` if you want to target the build machine.

3 The two stages

`lassip` runs in two stages, and most of the options belong to one or the other.

Stage 1 (`--calc-spec`, `--hapstats`, or both) reads genotypes and writes one row per window. `--calc-spec` writes the top K haplotype frequencies per window, which is the input to stage 2. `--hapstats` writes H12 and H2/H1, or G123 and G2/G1 for unphased data. Either can be used alone.

Stage 2 (`--lassi`, `--salti`, or `--avg-spec`) reads the spectra written by stage 1 and computes a composite likelihood ratio per window against a genome-wide null spectrum.

Options belonging to the other stage are accepted and ignored, with a warning naming them. Nothing is silently dropped.

4 Input files

4.1 VCF

One or more VCF files, given to `--vcf`, phased by default or read as multilocus genotypes with `--unphased`. Genotypes must be coded 0 and 1; missing calls (.) are allowed and are handled as described in section 6.

Several contigs may be analysed in one run, given either as one file per contig, as a single file holding several, or as a mixture. A run writes one spectra file per population covering every contig it was given. Two rules apply: no contig may appear twice across the input, and a file's records must be grouped by contig — lassip reads a VCF in one pass and will not buffer contigs in order to reassemble them, so sort by position first.

Contigs are processed one at a time and each one's genotypes are released before the next is read, so peak memory depends on how many contigs there are rather than on how they are packaged.

4.2 Population file

`--pop` takes a two column file, `<individual ID> <population ID>`, with no header. Only individuals listed there are analysed, and every listed individual must be present in every VCF given. When more than one population is present, all statistics are computed per population.

4.3 Genetic map

`--map` takes `<chr> <locus ID> <genetic position> <physical position>` and is read only by stage 2, and only under `--dist-type cm`. Windows whose midpoint falls outside the map, or in a gap wider than `--max-gap`, cannot be placed; lassip reports the first such window and stops rather than interpolating across the gap.

5 Statistics

All of the statistics implemented in LASSI-Plus concern the ranked haplotype frequency spectrum. Define it as $\mathbf{h} = (h_1, h_2, \dots, h_N)$, where h_i is the frequency of the i th most common haplotype in a region, N is the total number of unique haplotypes, and $h_1 \geq h_2 \geq \dots \geq h_N$. H12 (Garud et al. 2015) is then

$$H_{12} = (h_1 + h_2)^2 + \sum_{i=3}^N h_i^2,$$

and H2/H1 is

$$H_{2/1} = \sum_2^N h_i^2 \bigg/ \sum_1^N h_i^2.$$

Harris et al. (2018) construct the analogous statistics for unphased data from the ranked multilocus genotype frequency spectrum $\mathbf{g} = (g_1, g_2, \dots, g_N)$, with g_i the frequency of the i th most common multilocus genotype and $g_1 \geq g_2 \geq \dots \geq g_N$:

$$G_{123} = (g_1 + g_2 + g_3)^2 + \sum_{i=4}^N g_i^2, \quad G_{2/1} = \sum_2^N g_i^2 / \sum_1^N g_i^2.$$

These are what `--hapstats` computes: H12 and H2/H1 for phased data, and G123 and G2/G1 under `--unphased`.

LASSI (`--lassi`; Harris and DeGiorgio 2020) is a composite likelihood ratio comparing each window’s truncated haplotype frequency spectrum against a genome-wide null spectrum, under a model in which a sweep converts the frequency of the m most common haplotypes. It reports m and the statistic T .

saltiLASSI (`--salti`; DeGiorgio and Szpiech 2022) is an extension of LASSI that adds a spatial component to the composite likelihood ratio, using the decay of the distortion away from a putative sweep centre and taking flanking windows within `--max-extend-bp`, `--max-extend-cm` or `--max-extend-nw` into the likelihood. It reports the test statistic Λ — written L in the output columns — together with m and the decay rate A . Because it reads flanking windows, contig boundaries matter: they are taken from the `chr` column of the spectra file, so a file holding several contigs and one file per contig give identical results.

`--avg-spec` writes the average spectrum across the input, which is what `--null-spec` then reads if you want to supply the null explicitly rather than letting lassip compute it from the data it is given.

6 Missing data

A haplotype with a missing call in a window cannot be compared to other haplotypes by exact identity, so lassip groups haplotypes that are *compatible* — equal at every site observed in both — and the choice of grouping rule changes the spectrum. `--hap-cluster` selects it.

best-comp (default) assigns each incomplete haplotype to the most frequent haplotype compatible with it. Deterministic, and the most accurate of the three at low to moderate missingness.

garud-shuffle is the rule used by v1.2.2 and earlier: the distinct haplotypes are shuffled, and each in turn seeds a group that absorbs every not yet absorbed haplotype compatible with it. Which group an incomplete haplotype joins therefore depends on the shuffle. Kept for reproducing earlier results.

soft-em splits each incomplete haplotype fractionally across the haplotypes compatible with it, in proportion to their estimated frequencies. Experimental: class sizes come out fractional rather than as counts.

`--match-tol t` allows haplotypes differing at up to and including t observed sites to be grouped; the default 0 requires equality at every site observed in both. Note that the semantics changed in v1.3.0: the test was previously $< t$, so each setting behaved as the one below it.

`--max-lmiss` drops loci with too high a missing fraction and `--max-hmiss` leaves a haplotype out of a window's spectrum when too much of that window is missing for it.

7 Reproducibility

A run is determined by its inputs, its options and `--seed`. It does not depend on `--threads`, on window order, or on when it was run.

`--seed` (default 1) affects `--hap-cluster garud-shuffle` only, which is the one rule whose result depends on the order haplotypes are visited; the other two are deterministic and lassip warns if you set a seed with them. The seed is mixed with each window's SNP boundaries, so a window's clustering is the same whichever thread computes it. `--seed 0` restores the clock seeding used by v1.2.2 and earlier, which is useful only for examining how much the clustering varies between runs.

Note that the seed makes the order reproducible; it does not remove the order dependence. Which haplotype an incomplete one joins under `garud-shuffle` is still an arbitrary choice among compatible candidates.

8 Command line options

```
lassip --vcf <file> --pop <file> --calc-spec [--hapstats] \  
      --winsize <int> --winstep <int> --out <prefix>  
  
lassip --spectra <file> [<file> ...] (--lassi | --salti | --avg-spec) \  
      --out <prefix>
```

8.1 General

`--threads <int>`

The number of threads to spawn during computations.

Default: 1

`--seed <int>`

Seed for the shuffle used when clustering haplotypes that carry missing data. Runs with the same seed, inputs and flags are reproducible regardless of `--threads`, operating system or compiler. Set 0 to seed from the system clock instead, which reproduces the non-reproducible behaviour of lassip $\leq 1.2.2$.

Default: 1

8.2 Input and output

`--out <string>`

The basename for all output files.

Default: outfile

`--map <string>`

A map file formatted `<chr#> <locusID> <genetic pos> <physical pos>`. Sites in VCF not in map

file will be interpolated.

Default: (empty)

--vcf <string1> ... <stringN>

One or more VCF files containing haplotype data. Variants should be coded 0/1 and every file must contain all of the samples named by **--pop**. Several contigs may be analysed in one run, given either as one file per contig or as files holding several; the run then writes a single spectra file per population covering all of them. A contig may not appear twice, and a file's records must be grouped by contig.

Default: (empty)

--pop <string>

A file containing <ind ID> <pop ID>.

Default: (empty)

--spectra <string1> ... <stringN>

A list of spectra files for finalization. Contigs are delimited by the chr column rather than by the file, so a file may hold any number of them and the results are the same either way. A file's rows must be grouped by contig and ascending within one, and no contig may appear twice across the files given here.

Default: (empty)

--unphased <bool>

Set this flag to indicate data are unphased.

Default: false

8.3 Windows

--winsize <int>

The window size within which to calculate statistics.

Default: 0

--winstep <int>

The sliding window step size.

Default: 0

8.4 Statistics

--calc-spec <bool>

Set this flag to compute K-truncated haplotype frequency spectra.

Default: false

--avg-spec <bool>

Set this flag to compute and output the average K-truncated haplotype frequency spectrum from a set of .spectra files.

Default: false

--null-spec <string>

A file containing a null K-truncated haplotype spectrum for use computing LASSI or saltiLASSI.

Default: (empty)

--lassi <bool>

Set this flag to use the LASSI method.

Default: false

--lassi-choice <int>

Set this flag to change the way LASSI distributes mass across sweeping haplotype classes. Takes an integer in {1..5}.

Default: 4

--hapstats <bool>

Set this flag to calculate haplotype statistics.

Default: false

--salti <bool>

Set this flag to use the saltiLASSI method.

Default: false

--k <int>

Top K haplotypes for LASSI computations.

Default: 10

8.5 Filtering

--filter-level <int>

Filter monomorphic sites and sites with missing data: 0-no filtering, 1-compute freq for all samples, 2-compute freq per pop.

Default: 2

--max-lmiss <double>

Filter loci with > this proportion of missing data.

Default: 0.10

--max-hmiss <double>

Drop haplotypes with > this proportion of missing data when computing the HFS.

Default: 0.20

--hap-cluster <string>

How haplotypes carrying missing genotypes are grouped into classes. Missing sites act as wildcards, so such a haplotype can be compatible with several distinct haplotypes and something has to choose. One of: best-comp (default) each haplotype joins the most frequent class it is compatible with, taking the best-observed haplotypes first. Deterministic; **--seed** has no effect on it. garud-shuffle the pre-1.3 behaviour: shuffle the haplotypes, then let each in turn absorb every compatible one not yet claimed. The result depends on the shuffle, so runs differ unless **--seed** is fixed. soft-em EXPERIMENTAL. Divide an ambiguous haplotype's count across the classes it could belong to in proportion to their frequencies, iterated to convergence. Class sizes become fractional. Without missing genotypes and at **--match-tol** 0 all three give the same classes.

Default: best-comp

--match-tol <int>

Group haplotypes with missing data into the same class as a haplotype with no missing data if they

have $\leq$ this many pairwise differences.

Default: 0

--keep-monomorphic <bool>

Set this flag to retain any monomorphic sites in the data.

Default: false

8.6 saltiLASSI

--dist-type <string>

Distance measure for saltiLASSI: bp, cm, nw.

Default: bp

--max-gap <double>

Widest gap in basepairs between two genetic map positions that **--dist-type** cm will interpolate across. A window falling in a wider gap cannot be placed and lassip stops rather than guess its genetic position. Real maps have gaps at centromeres and other low-recombination regions, so raise this if your map is sparse where your data are dense. Set 0 to interpolate across any gap.

Default: 3000000.00

--max-extend-bp <double>

Maximum distance in basepairs from core window to consider for saltiLASSI.

Default: 100000.00

--max-extend-cm <double>

Maximum distance in centimorgans from core window to consider for saltiLASSI.

Default: 0.05

--max-extend-nw <double>

Maximum distance in number of windows from core window to consider for saltiLASSI.

Default: 5.00

9 Output files

Stage 1 writes <out>.lassip.[hap|mlg].spectra.gz when **--calc-spec** is set and <out>.lassip.[hap|mlg].stats when only **--hapstats** is. hap is used for phased data and mlg for **--unphased**. At the default **--filter-level** 2 each population is filtered separately and gets its own file, named <out>.<pop>.lassip.[hap|mlg].

Stage 2 writes a single <out>.lassip.[hap|mlg].out.gz.

The stats file has one row per window:

```
<chr> <start> <end> <nSNPs> <ppos> <nHaps> <uniqHaps> <h12|g123> <h2h1|g2g1>
```

The spectra file has a header line and then one row per window:

```
#phased <0|1> hapstats <0|1> wins <N> K <K> npop <P> <pop1> ... <popP> \  
[contigs <C> <name> <rows> ...]
```



```
<chr> <start> <end> <nSNPs> <ppos> <nHaps> <uniqHaps> \
    [<h12|g123> <h2h1|g2g1>] <hfs_1> ... <hfs_K>
```

`wins` is the number of window rows. The `contigs` field is written only when a file holds more than one contig, and lists each contig with the number of rows it contributes; stage 2 validates it against the rows. It sits after the population names, so a reader that parses the header positionally stops before it — which also means lassip 1.2.2 and earlier will read a multi-contig file written by this version without complaint and analyse every contig in it as one block. Split such files by contig before sharing them with an older install. Files written by older versions carry no `contigs` field and are read exactly as before.

Stage 2 writes the same leading columns, naming the position column `pos` rather than `ppos`, followed by `<m> <T>` for `--lassi` and `<m> <A> <L>` for `--salti`.

In every case the per-population columns are repeated for each population, with the population code prepended to the column name. `hfs_N` is the frequency of the N th most common haplotype in the window, and `ppos` is the window's physical midpoint — or the window's index within its contig when `--dist-type nw` was used, which is why that mode's positions are only meaningful contig by contig.

10 Exit codes

lassip follows the `sysexits.h` conventions, so a script can tell the kinds of failure apart without parsing `stderr`.

code	meaning
0	success, and for <code>--help</code> and <code>--version</code>
64	the command line is wrong: an unknown option, a missing required option, a value out of range, or an option used at the wrong stage
65	an input file opened but holds the wrong thing: a malformed record, contigs that are not grouped, a contig given twice, a genetic map that does not cover the data, a spectra file disagreeing with its own header
70	an internal error
74	a file could not be opened: a missing input, or an output path that cannot be created

The distinction between 65 and 74 is about opening, not about the option: `--map missing.map` exits 74, while `--map a-map-for-another-contig.map` exits 65.

11 Examples

A single contig, both stages:

```
lassip --vcf YRI.chr22.vcf.gz --pop YRI.ids.pop.txt --calc-spec --hapstats \
    --k 10 --winsize 117 --winstep 12 --out YRI.chr22
```

```
lassip --spectra YRI.chr22.YRI.lassip.hap.spectra.gz --salti \
      --dist-type bp --max-extend-bp 100000 --out YRI.chr22
```

Note the population name in the second command's filename: at the default `--filter-level 2` stage 1 writes one file per population.

A whole genome in one run, as one file per chromosome:

```
lassip --vcf chr1.vcf.gz chr2.vcf.gz ... chr22.vcf.gz --pop pops.txt \
      --calc-spec --hapstats --k 10 --winsize 117 --winstep 12 \
      --threads 8 --out scan
```

```
lassip --spectra scan.YRI.lassip.hap.spectra.gz --salti \
      --dist-type bp --max-extend-bp 100000 --threads 8 --out scan
```

The same thing from a single multi-contig VCF is `--vcf genome.vcf.gz`, and gives identical results. Passing the per-contig spectra files separately to stage 2 also gives identical results; what you must not do is concatenate them by hand, because the rows of one contig then sit inside another's block.

Unphased data, where G123 and G2/G1 replace H12 and H2/H1 and the output files are named `mlg` rather than `hap`:

```
lassip --vcf genome.vcf.gz --pop pops.txt --unphased --calc-spec --hapstats \
      --k 10 --winsize 117 --winstep 12 --out scan
```

`example/do_lassip_YRI.bash` runs the first of these against the data in `example/` and reproduces the outputs committed beside it.

12 References

saltiLASSI

DeGiorgio M, Szpiech ZA (2022) A spatially aware likelihood test to detect sweeps from haplotype distributions. *PLoS Genetics* 18:e1010134.

LASSI

Harris AM, DeGiorgio M (2020) A likelihood approach for uncovering selective sweep signatures from haplotype data. *Molecular Biology and Evolution* 37:3023–3046.

H12 and H2/H1

Garud NR, Messer PW, Buzbas EO, Petrov DA (2015) Recent selective sweeps in North American *Drosophila melanogaster* show signatures of soft sweeps. *PLoS Genetics* 11:e1005004.

G123 and G2/G1

Harris AM, Garud NR, DeGiorgio M (2018) Detection and classification of hard and soft sweeps from unphased genotypes by multilocus genotype identity. *Genetics* 210:1429–1452.

Changes

The change log is in the README.